

“초등부 2번 / 중등부 1번. 거울” 문제 풀이

작성자: 반딧불

부분문제 1

$N = 1$ 인 경우, 답은 $2A_1 - s$ 가 된다.

$N = 2$ 인 경우, 거울을 사용하는 순서는 총 두 가지가 있다. 1번 거울을 먼저 사용하는 경우 최종 위치는 $2A_2 - (2A_1 - s)$ 이고, 2번 거울을 먼저 사용하는 경우 최종 위치는 $2A_1 - (2A_2 - s)$ 이다. 둘 중 더 큰 것을 출력하면 된다.

부분문제 2

부분문제 1의 상황에서 2번 거울을 이용하는 것이 더 최적일 조건을 분석하면 $2A_1 - 2A_2 + s \geq 2A_2 - 2A_1 + s$, 즉 $A_1 \leq A_2$ 이다. 따라서 이용하는 거울이 두 개뿐일 때는 더 오른쪽에 있는 거울을 나중에 사용하는 것이 이득이다.

또한, 두 거울을 이용한 뒤의 위치 공식의 형태에서부터, 초기 위치가 오른쪽에 있을 수록 두 개의 거울을 이용한 뒤의 최종 위치도 오른쪽에 있다는 사실을 알 수 있다.

부분문제 2의 상황에서는 이용할 수 있는 거울의 위치가 단 두 가지뿐이다. 왼쪽 거울의 위치 A_1 을 L 이라고 하고, 오른쪽 거울의 위치 A_N 을 R 이라고 하자. 두 가지 종류의 거울 중 적당히 골라 두 번의 위치로 가장 오른쪽으로 보내기 위해서는 먼저 위치 L 의 거울을, 그 다음으로 위치 R 의 거울을 이용하는 것이 최적이다. (초기 위치가 s 일 때 최종 위치가 $2A_2 - 2A_1 + s$ 이므로, A_1 은 작을 수록, A_2 는 클 수록 최적이다.)

따라서 거울을 이 두 종류만 이용할 수 있을 때, 홀수 번째에는 왼쪽 거울을, 짝수 번째에는 오른쪽 거울을 이용하는 것이 최적이다. 마침 두 거울이 정확히 $N/2$ 종류씩 존재하므로, 이 경우가 답이 된다. 이 과정을 직접 시뮬레이션해 답을 $O(N)$ 에 구할 수 있다.

부분문제 3

N 이 짝수이다. N 개 거울의 위치를 이용하는 순서대로 $x_1, x_2, \dots, x_N$ 이라고 하자. 이때 초기 위치 s 에 대해 최종 위치를 식으로 쓰면 다음과 같다.

- 1번 거울까지 이용했을 때: $2x_1 - s$
- 2번 거울까지 이용했을 때: $2x_2 - (2x_1 - s) = 2x_2 - 2x_1 + s$
- 3번 거울까지 이용했을 때: $2x_3 - (2x_2 - 2x_1 + s) = 2(x_1 + x_3) - 2x_2 - s$
- 4번 거울까지 이용했을 때: $2x_4 - (2(x_1 + x_3) - 2x_2 + s) = 2(x_2 + x_4) - 2(x_1 + x_3) + s$
- ...
- N 번 거울까지 이용했을 때: $2(x_2 + x_4 + \dots + x_N) - 2(x_1 + x_3 + \dots + x_{N-1}) + s$ (단, N 은 짝수)

식의 형태를 통해, 홀수 번째에 해당하는 $N/2$ 개의 턴에는 왼쪽에 있는 $N/2$ 개의 거울을 이용하고, 짝수 번째에 해당하는 $N/2$ 개의 턴에는 오른쪽에 있는 $N/2$ 개의 거울을 사용하는 것이 최적임을 알 수 있다. (홀수 번째 턴끼리, 또는 짝수 번째 턴끼리 거울을 바꾸는 것은 최종 결과에 영향이 없음을 알 수 있다.)

거울의 위치가 정렬되어 주어지므로, 답을 $O(N)$ 에 구할 수 있다.

이외에 $A_{N/2} < s < A_{N/2+1}$ 이라는 점을 이용하는 다른 풀이가 있을 수 있다.

부분문제 4

부분문제 3에서 N 이 짝수인 경우의 풀이를 설명하였다. N 이 홀수인 경우, 사용한 거울의 위치 $x_1, x_2, \dots, x_N$ 에 대해 최종 위치는 $2(x_1 + x_3 + \dots + x_N) - 2(x_2 + x_4 + \dots + x_{N-1}) - s$ 가 된다.

식의 형태를 통해 홀수 번째에 해당하는 $(N+1)/2$ 개의 턴에는 오른쪽에 있는 $(N+1)/2$ 개의 거울을, 짝수 번째에 해당하는 $(N-1)/2$ 개의 턴에는 왼쪽에 있는 $(N-1)/2$ 개의 거울을 사용하는 것이 최적임을 알 수 있다.

거울의 위치가 정렬되어 주어지므로, 답을 $O(N)$ 에 구할 수 있다.

“중등부 2번. 가방” 문제 풀이

작성자: 구재현

부분문제 1

백트래킹을 통해서, 가방에 담을 물건들의 모든 경우의 수를 열거한다. 각 경우 S 에 대해서:

- 가방에 담기게 되는 물건의 무게 합 (A_S)
- 도둑이 남은 물건을 훔쳐갔을 때의 무게 합 (B_S)

를 계산할 수 있다. 모든 2^N 개의 경우에 대해서 이를 배열에 저장해 두자. 각 x 에 대한 문제의 정답은, $A_S \leq x$ 인 S 중 B_S 의 최댓값이 된다. 시간 복잡도는 $O(2^N \cdot C)$ 이다.

부분문제 2

먼저 입력으로 주어진 모든 물건들을 무게 오름차순으로 정렬한다. 이 경우, 도둑은 내가 고르지 않은 물건들 중 맨 앞에 등장하는 최대 K 개의 물건을 훔칠 것이다.

동적 계획법을 사용한다. $dp[i][j][w] =$ (첫 i 개의 물건에 대해서 가방에 넣을지 말지를 결정했으며, 도둑이 j 개의 물건을 들고 갔고, 지금까지 가방에 들어간 물건의 무게 합이 w 이하일 때, 도둑이 훔쳐가는 물건 무게의 최댓값) 으로 정의하면:

- $i+1$ 번째 물건을 가방에 넣는 경우: $dp[i+1][j][w + A_{i+1}]$ 로 상태 전이한다.
- $i+1$ 번째 물건을 가방에 넣지 않는 경우: 만약에 도둑이 K 개 미만의 물건을 훔쳐갔다면, $dp[i+1][j+1][w]$ 로 전이하며, A_{i+1} 만큼의 무게를 도둑이 훔쳐간다고 추가한다. 그렇지 않은 경우, $dp[i+1][j][w]$ 로 전이하며, 도둑이 추가로 훔쳐가는 무게는 없다.

각 x 에 대해, $i = N, 0 \leq j \leq K, w = x$ 인 경우 중 $dp[i][j][w]$ 의 최댓값을 계산하면 된다. 시간 복잡도는 $O(N^2 \cdot C)$ 이다.

부분문제 3

물건이 무게 순으로 정렬되었고 도둑은 가벼운 물건부터 훔쳐간다. 따라서, 어떠한 인덱스 $0 \leq i \leq N$ 에 대해, 도둑은 인덱스 i 이하의 물건 중 가방에 들어있지 않은 물건을 모두 훔쳐가고, 인덱스 i 초과를 물건을 훔치지 않는다. 이 인덱스 i 를 고정시켰을 때 도둑이 훔쳐가는 물건의 무게 합을 최대화하려면:

- $1, 2, \dots, i$ 번 물건 중 정확히 $i - K$ 개의 물건을 가방에 담아야 한다.
- 가방에 담는 물건의 무게는 x 이하여야 하며, 가능한 작아야 한다.

만약에 위 조건을 만족하게끔 $w \leq x$ 만큼의 물건을 가방에 담을 수 있었다면, 도둑은 $\sum_{j=1}^i A_j - w$ 만큼을 훔쳐가게 된다.

정확히 $i - K$ 개의 물건을 담는다는 조건은 $i - K$ 개 이상의 물건을 담는다는 조건으로 바꿀 수 있다. 도둑이 인덱스 i 초과를 물건을 가져가는 경우도 고려되지만, 해당 경우는 실제 도둑이 훔쳐가게 될 무게 이하의 값이 고려되게 되어서 문제가 되지 않는다.

이제 동적 계획법을 사용한다. $dp[i][w] = (\text{첫 } i \text{ 개의 물건에 대해서 가방에 넣을지 말지를 결정했으며, 지금까지 가방에 들어간 물건의 무게 합이 } w \text{ 이하일 때, 가방에 있는 물건의 개수 최대})$ 로 정의하자. 각 x 에 대해서, $0 \leq i \leq N, w = x$ 인 경우 중 $dp[i][w]$ 의 최댓값을 계산하면 된다. 시간 복잡도는 $O(N \log N + NC)$ 이다.

부분문제 4

도둑이 훔쳐가는 물건이 단 하나이다. 도둑이 $1 \leq i \leq N$ 번 물건을 훔쳐가게 하기 위해서는 $\sum_{1 \leq j \leq i-1} A_j$ 가 x 이하여야 한다. 이를 만족하는 최대 i 를 $x = 1, 2, \dots, C$ 에 대해 모두 구해주면 된다. x 가 증가함에 따라 최대 i 도 증가하니, 현재까지의 최대 i 와 $\sum_{1 \leq j \leq i-1}$ 을 관리해 주면 (two pointers) 문제를 $O(N \log N + C)$ 에 해결할 수 있다.

부분문제 5

전체 문제는 그리디 알고리즘을 사용하여 해결할 수 있다. 다음 두 가지를 관찰하자:

- $N - K$ 개 초과를 물건을 가져갈 필요는 없다: 추가로 가져간 물건을 가방에서 빼면, 도둑이 해당 물건을 가져가게 된다.
- i 번 물건을 가져가지 않고 $i + 1$ 번 물건을 가져갈 필요는 없다: $i + 1$ 번 물건 대신 i 번 물건을 가방에 담으면, 가방의 무게는 그대로거나 감소하고, 도둑이 가져가는 물건의 무게는 그대로거나 증가한다. (i 번 물건이 들어갈 자리에 $i + 1$ 번 물건이 들어감)

두 조건에 의해서, 우리가 가져와야 할 물건은 가장 가중치가 낮은 i ($0 \leq i \leq N - K$) 개의 물건임을 알 수 있다. 이러한 물건을 가져가면, 도둑은 $i + 1, i + 2, \dots, i + K$ 번 물건을 가져가게 된다. 즉, 각 x 에 대해 $\sum_{1 \leq j \leq i-1} A_j$ 가 x 이하인 최소 i 를 구하고, $i + 1, i + 2, \dots, i + K$ 번 물건의 가중치 합을 출력하면 된다. 앞 부분은 부분문제 4와 동일하고, 뒷 부분은 부분합을 사용하면 된다. 전체 문제를 $O(N \log N + C)$ 에 해결할 수 있다.

“중등부 3번. 로봇” 문제 풀이

작성자: 오주원

부분문제 1

$N = 1$ 이므로 점프대가 하나뿐이다. 로봇은 왼쪽으로 이동하다가 이 점프대에 도착하면 점프대를 작동시키는 것을 반복한다. 다만 초기에 로봇이 점프대보다 왼쪽에 있는 경우에는 로봇이 점프대에 도달할 수 없으므로, 단순히 왼쪽으로 T 만큼 이동한 위치가 정답이 된다.

점프대는 작동될 때마다 파워가 두 배로 증가하므로, 동일한 점프대를 이용할 수 있는 횟수는 최대 $O(\log T)$ 번이다. 또한 점프대가 등장하기 전까지 왼쪽으로 이동하는 횟수는 단순히 현재 위치와 점프대의 위치 차이를 계산하여 $O(1)$ 에 구할 수 있다.

따라서, 로봇의 이동을 시뮬레이션할 때 왼쪽으로의 단순 이동은 한 번의 계산으로 처리하고, 점프대에 도착했을 때 점프와 파워를 증가시키는 행동을 반복하면 된다. 이 방법을 사용하면 로봇의 최종 위치를 $O(\log T)$ 시간에 구할 수 있다.

부분문제 2

부분문제 1과 달리 점프대가 두 개이므로 두 점프대 사이에서 출발하는 경우를 고려해야 한다. 특히 왼쪽 점프대를 사용하다가 점프 후 오른쪽 점프대의 위치를 넘어가는 상황까지 처리할 수 있도록 구현해야 한다.

부분문제 3

로봇이 이동하는 경로는 지문의 예시와 같은 방식으로 그대로 시뮬레이션할 수 있다. 매 초마다 현재 위치에 점프대가 존재하는지를 확인하고, 이에 따라 이동을 결정하면 된다. 각 초마다 점프대의 존재 여부를 확인하는 데 $O(N)$ 의 시간이 걸리므로, 전체적으로 T 초 동안의 시뮬레이션은 $O(NT)$ 의 시간복잡도로 수행할 수 있다. 따라서 전체 시간복잡도는 $O(QNT)$ 이다.

부분문제 4

부분문제 3과 마찬가지로 로봇이 이동하는 경로는 지문의 예시처럼 그대로 시뮬레이션할 수 있다. 다만 매 초마다 현재 위치에 점프대가 존재하는지를 단순히 $O(N)$ 에 탐색하는 방식은 비효율적이다. 이를 개선하기 위해 정렬된 점프대의 위치에서 이분 탐색을 사용하면 한 번의 탐색을 $O(\log N)$ 에 수행할 수 있다. 혹은 배열이나 set과 같은 자료구조를 사용하여 점프대의 존재 여부를 $O(1)$ 또는 $O(\log N)$ 에 확인하도록 구현할 수도 있다. 이렇게 하면 전체 시간복잡도를 $O(QT \log N)$ 혹은 $O(QT)$ 로 줄일 수 있다.

부분문제 5

어떤 점프대에서 점프를 막 시작해서 다음 점프대의 위치를 넘어가 그 점프대에 도달하여 새로운 점프대를 사용하게 되는 과정을 **점프대 이동**이라고 정의하자. 이러한 **점프대 이동**은 최대 $N - 1$ 번만 일어날 수 있다.

따라서 부분문제 2에서와 같이, 다음 점프대로 넘어가는 과정을 반복하며 시뮬레이션하면 된다. 다만 i 번 점프대에서 **점프대 이동**이 발생했다고 해서 반드시 $i + 1$ 번 점프대로 이동하는 것은 아니므로, 이 점을 주의해야 한다.

각 점프대에서 점프 횟수는 최대 $O(\log T)$ 에 불과하며, 이러한 과정을 N 개의 모든 점프대에 대해 Q 개의 각 시작 위치에 반복하므로, 이 방법의 시간 복잡도는 $O(NQ \log T)$ 이다.

하지만 Q 개의 질의를 계산하기 전에, 각 점프대에 대해서 처음 도달한 이후 **점프대 이동이 발생하기까지 걸리는 시간**을 미리 전처리 해두면 더 빠른 시간으로 문제를 해결할 수 있다. 이 전처리는 각 점프대별로 최대 $O(\log 10^{17})$ 번의 시뮬레이션을 통해 $O(N \log 10^{17})$ 의 시간으로 필요한 값을 구할 수 있다. 이렇게 전처리가 끝나면, 각 시작 위치에 대한 계산에서 더 이상 점프 횟수를 일일이 시뮬레이션할 필요가 없으므로, 각 경우를 $O(N)$ 에 처리할 수 있어 전체 시간복잡도는 $O(NQ)$ 으로 줄어든다.

수식 정리를 통해 $O(1)$ 에 점프대 이동이 걸리는 시간을 계산하는 방법으로도 $O(NQ)$ 의 시간복잡도가 나오도록 최적화를 할 수 있다.

부분문제 6

$dp[i][j]$ 를 i 번째 점프대에 처음 도달한 이후 j 번의 **점프대 이동이 발생하기까지 걸리는 시간**이라고 정의하고, $nxt[i][j]$ 를 i 번째 점프대에서 j 번의 **점프대 이동이 발생한 이후의 도달하게 되는 점프대의 번호**로 정의하자.

이 값들은 부분문제 5의 사용한 방법으로 $dp[*][1]$ 와 $nxt[*][1]$ 을 먼저 구한 뒤, 점프대 번호 i 가 감소하는 순서대로 $dp[i][*]$ 와 $nxt[i][*]$ 를 채워 나가면 된다. 이를 통해 각 점프대에서 여러 번의 **점프대 이동**이 발생한 이후의 걸린 시간과 도달하는 점프대의 번호를 미리 구할 수 있다.

이 값들을 미리 구해놓으면 더 이상 매번 $O(N)$ 번의 점프대 이동을 직접 시뮬레이션할 필요가 없다. 로봇이 처음으로 도달하는 점프대의 번호를 k 라고 할 때, $dp[k][i]$ 값은 i 가 증가할수록 증가하므로, $dp[k][i]$ 가 T 이하인 최대 i 를 이분 탐색으로 $O(\log N)$ 에 찾을 수 있다. 이렇게 하면 **점프대 이동**이 더 이상 일어나지 않을 때까지의 과정을 스킵할 수 있으며, 이후 남은 시간 동안은 단일 점프대에서의 이동만 단순 시뮬레이션하면 된다.

전처리 과정에서 $dp[*][1]$ 와 $nxt[*][1]$ 을 구하는 데 $O(N \log 10^{17})$ 의 시간이 필요하며, $dp[*][*]$ 와 $nxt[*][*]$ 을 모두 구하는 데 $O(N^2)$ 의 시간이 소요된다. 전처리 이후에는 각 시작 위치에 대해 처음 도달하는 점프대를 찾는데 $O(\log N)$, 그리고 그 점프대에서 이분 탐색을 통해 최대 **점프대 이동** 횟수를 찾는 데 $O(\log N)$, 마지막으로 남은 시간동안 단일 점프대에서 시뮬레이션을 수행하는 데 $O(\log T)$ 가 걸린다. 따라서 전체 시간복잡도는 $O(N \log 10^{17} + N^2 + Q(\log N + \log T))$ 이다.

부분문제 7

$dp[i][j]$ 를 i 번째 점프대에 처음 도달한 이후 2^j 번의 **점프대 이동이 발생하기까지 걸리는 시간**이라고 정의하고, $nxt[i][j]$ 를 i 번째 점프대에서 2^j 번의 **점프대 이동이 발생한 이후의 도달하게 되는 점프대의 번호**로 정의하자.

우선 $dp[i][0]$ 과 $nxt[i][0]$ 은 부분문제 5에서 사용한 방식과 동일하게 초기화한다. 이후 j 가 증가하는 순서로 아래 점화식을 이용해 값을 채울 수 있다.

- $dp[i][j] = dp[i][j-1] + dp[nxt[i][j-1]][j-1]$
- $nxt[i][j] = nxt[nxt[i][j-1]][j-1]$

T 초 동안의 점프대 이동을 효율적으로 계산하기 위해서는, 매 단계에서 현재 남은 시간 안에서 최대한 많이 **점프대 이동**을 수행할 수 있도록 선택하는 것이 핵심이다. 현재 위치한 점프대가 k , 남은 시간이 T_{rem} 이라고 하자. $dp[k][j] \leq T_{rem}$ 인 j 중에서 가장 큰 j 를 찾아 이동하면 한 번에 2^j 번의 점프대 이동을 처리할

수 있으며, 이동 후에는 T_{rem} 에서 $dp[k][j]$ 를 빼고, 점프대 번호 k 를 $next[k][j]$ 로 갱신한다. 이 과정을 남은 시간이 허용하는 동안 반복하면, T 초 동안 발생하는 모든 점프대 이동을 빠르게 처리할 수 있고, 이후 남은 시간 동안은 단일 점프대에서의 이동만 $O(\log T)$ 의 시간으로 단순 시뮬레이션하면 된다.

이 방법은 $O(\log^2 N)$ 이 아닌 $O(\log N)$ 으로 구현할 수 있는데, 한 번의 선택에서 반드시 j 가 감소하는 방향으로 진행하는 것을 이용하면 된다. 만약 $dp[k][j]$ 를 선택한 뒤에도 j 이상인 어떤 j' 에 대해 $dp[k][j']$ 을 선택할 수 있었다면, 이는 처음부터 $dp[k][j+1]$ 을 선택했어야 하므로 모순이다. 따라서 j 는 매 단계마다 감소하며, 최대 $\log N$ 번의 선택으로 모든 **점프대 이동**을 처리할 수 있다.

이 방식에서 $dp[*][0]$ 와 $next[*][0]$ 을 구하는 데 $O(N \log 10^{17})$, $dp[*][*]$ 와 $next[*][*]$ 을 모두 채우는 데 $O(N \log N)$ 이 소요된다. 이후 각 시작 위치에 대해서는 $O(\log N + \log T)$ 만에 계산할 수 있으므로, 전체 시간복잡도는 $O(N \log 10^{17} + Q(\log N + \log T))$ 이다.

“중등부 4번 / 고등부 3번. 수열과 쿼리” 문제 풀이

작성자: 구재현

부분문제 1

쿼리가 주어졌을 때, i 를 고정하고 j 를 증가시키면서 모든 연속 구간을 열거하고 각각의 합을 계산할 수 있다. 이 중 최댓값을 출력하면 된다. 시간 복잡도는 $O(QN^2)$ 이다.

부분문제 2

배열 $S_i = \sum_{j=0}^{i-1} A_j$ 를 정의하자. 이는 점화식 $S_{i+1} = S_i + A_i - X$ 를 사용하여 $O(N)$ 에 계산할 수 있다.

쿼리 X 가 주어졌을 때, 문제의 정답은 $\max_{0 \leq j < i \leq N} (S_i + iX - S_j - jX) = \max_{1 \leq i \leq N} (S_i + iX - (\min_{0 \leq j \leq i-1} S_j + jX))$ 이다. 배열 $D_i = \min_{j \leq i} (S_j + jX)$ 를 정의하자. 이는 점화식 $D_{i+1} = \min(D_i, S_{i+1} + (i+1)X)$ 를 사용하여 $O(N)$ 에 계산할 수 있다. 문제의 정답은 $\max_{1 \leq i \leq N} (S_i + iX - D_{i-1})$ 이고, $O(N)$ 에 계산할 수 있다. 시간 복잡도는 $O(QN)$ 이다.

부분문제 3

M_l 을 길이 l 의 부분 배열 중 합의 최댓값이라고 정의하자. 배열 M 은 $O(N^2)$ 시간에 계산할 수 있다. 이제 문제의 정답은 $\max_{l=1}^N (M_l + lX)$ 이다. 이를 단순히 계산하면 쿼리당 $O(N)$ 시간이 걸리니, 최적화하자.

2차원 평면 상에 N 개의 점 $(x_l, y_l) = (l, M_l)$ 을 그렸을 때, 쿼리 P 에 대해 우리는 $x_l \cdot P + y_l$ 의 최댓값을 계산하고 싶다 (입력에는 X 로 주어지지만 이제부터 혼동을 막기 위해 P 로 표현한다). N 개의 점들로 볼록 껍질 (convex hull) 을 구했을 때, 최댓값이 얻어지는 점은 볼록 껍질의 위쪽에 기울기 $-P$ 의 직선을 그었을 때의 접점임을 알 수 있다. 이러한 접점은, 볼록 껍질의 위 부분 (upper convex hull) 을 구하고, 각 직선마다 이분 탐색을 사용하여 $O(\log N)$ 시간에 찾을 수 있다. 시간 복잡도는 $O(N^2 + Q \log N)$ 이다.

부분문제 6

배열 M 을 빠르게 구하는 것은 어렵기 때문에, 다른 접근이 필요하다.

부분문제 3의 알고리즘과 동치이지만 약간 더 비효율적인, 다음과 같은 알고리즘을 생각하자: 2차원 평면 상에 $N(N+1)/2$ 개의 점 $\{(i-j, S_i - S_j)\}_{0 \leq j < i \leq N}$ 이 주어졌을 때, 쿼리 P 에 대해 $x_l \cdot P + y_l$ 의 최댓값이 문제의 정답이 된다. 모든 $N(N+1)/2$ 개의 점들을 가지고 upper convex hull을 구한 이후, 부분문제 3과 똑같은 이분 탐색을 사용하면 각 쿼리를 $O(\log N)$ 시간에 계산할 수 있다. 하지만, 점의 개수가 너무 많기 때문에, 컨벡스 헐을 구하는 것이 쉽지 않아 보인다.

이 문제를 분할 정복으로 해결하자. $f(L, R)$ 을, 점 집합 $\{(i-j, S_i - S_j)\}_{L \leq j < i \leq R}$ 의 upper convex hull을 반환하는 함수로 정의하자. $M = (L+R)/2$ 로 정의하면, $f(L, M), f(M+1, R)$ 을 호출하여 $i \leq M, j \geq L+1$ 인 경우의 컨벡스 헐의 후보를 모두 얻을 수 있다. 따라서, 점 집합 $\{(i-j, S_i - S_j)\}_{L \leq j \leq M < i \leq R}$ 의 upper convex hull을 계산하고, 재귀적으로 계산한 컨벡스 헐의 후보랑 합쳐준 다음에, 해당 점들로 upper convex hull을 다시 계산한 후 반환하면 된다.

어떠한 점 $(i-j, S_i - S_j)$ 가 upper convex hull에 있다는 것은, 기울기 p 가 존재해서 $(i-j)p + S_i - S_j$ 의 최댓값이 해당 점에서 유일하게 얻어진다는 것과 동치이다. (다시 말해, 모든 다른 점들은 $(i-j)p + S_i - S_j$ 의 값이 해당 점보다 작다). 이를 위해서는 $i \in [M+1, R]$ 에서 $S_i + ip$ 가 최대여야 하고, $j \in [L, M]$ 에서 $S_j + jp$ 가 최소여야 한다. 이는, $M+1 \leq i \leq R$ 에 있는 점 (i, S_i) 중 upper convex hull에 있는 점들만

고려해도 되고, $L \leq j \leq M$ 에 있는 점 (j, S_j) 중 lower convex hull에 있는 점들만 고려해도 된다는 것과 동치이다.

$\{(i, S_i)\}_{i=M+1}^R$ 에 있는 점들로 upper convex hull을 구하게 되면, 컨벡스 헐 상에서 인접한 점들을 사용하여 각 점이 최댓값을 이루는 p 의 범위를 계산할 수 있다. 같은 방식으로, $\{(j, S_j)\}_{j=L}^M$ 에 있는 점들로 lower convex hull을 구하게 되면, 각 점이 최솟값을 이루는 p 의 범위를 계산할 수 있다. 위의 논리로 이 p 의 범위가 겹치는 (i, j) 들의 쌍들만 계산하면 되고, 이는 머지 소트에서 병합하는 과정과 유사한, 아주 짧은 코드로 해결할 수 있다. (이러한 계산을 Minkowski Sum이라고 부른다.)

분할 정복 알고리즘은 컨벡스 헐을 구하는 과정에서 정렬을 필요로 하기 때문에 $T(N) = 2T(N/2) + O(N \log N) = O(N \log^2 N)$ 시간을 필요로 한다. 고로 시간 복잡도는 $O(N \log^2 N + Q \log N)$ 이다.

부분문제 7

위 분할 정복을 $O(N \log N)$ 에 구현하면 된다. 먼저, $\{(i, S_i)\}_{i=M+1}^R$ 에 있는 점들로 upper convex hull을 구할 때, 자연스럽게 각 점을 i 가 증가하는 순서대로 본 후 Monotone Chain을 사용하면 $O(N)$ 시간에 upper convex hull을 계산할 수 있다. lower convex hull도 동일하다. 또한, $f(L, R)$ 이 정확히 upper convex hull을 반환하게끔 구현할 필요가 없다: 병합 과정에서 나온 점들을 아무 순서 없이 모은 후 마지막에 한번 upper convex hull을 계산해 주면 된다. 최종적으로, 쿼리 처리 직전에 upper convex hull을 계산해 줄 때, 각 점의 x 좌표가 $[1, N]$ 범위이기 때문에, 단순히 각 x 좌표에 대한 y 좌표 최댓값을 배열에 기록해 주는 방식으로 정렬을 대체할 수 있다. 시간 복잡도는 $O((N + Q) \log N)$ 이다.

Remark 1. 입력 제한이 커서, 컨벡스 헐 등을 CCW를 사용하여 계산하였을 때 long long 범위를 초과하는 문제가 발생할 수 있다. 이 문제는 네 가지 서로 다른 방법으로 해결할 수 있다 (편리성 증가하는 순으로 나열). 모범 코드는 두 번째 방법으로 구현되었다.

- CCW 대신 기울기를 비교하는 함수를 사용하자. 쿼리 값이 모두 정수이기 때문에, 기울기를 정확한 실수가 아니라 정수 몫을 취한 값으로 부정확하게 비교하여도 문제가 없다.
- CCW의 값을 double 자료형으로 계산한 후, 절댓값이 10^{18} 이하일 경우 long long 으로 (오버플로우가 나도록) 다시 정확히 계산.
- CCW의 값을 long double 자료형으로 계산.
- CCW의 값을 __int128 자료형으로 계산: 이번 대회에 사용된 Biko의 채점 환경은 이를 지원한다.

Remark 2. 이 문제에 대해 기존에 알려진 풀이는 Sakai의 2024년 논문으로 [Sak24], $O(N \log^2 N)$ 의 전처리 시간이 걸린다. 해설의 풀이는 동일한 공간 복잡도와 쿼리 시간을 유지하며, 전처리 시간을 $O(N \log N)$ 으로 개선한다. 또한, 해설의 풀이는 논문의 접근과 비교했을 때 훨씬 단순하다.

References

- [Sak24] Yoshifumi Sakai. A data structure for the maximum-sum segment problem with offsets. In 35th Annual Symposium on Combinatorial Pattern Matching (CPM 2024), pages 26–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.